The Invisible Prosodist: The Tokenizer as an Unacknowledged Author of Machine Verse

Wenhao Lu
NXT Horizon
wenhao@nxthorizon.org

Abstract

Every poem a language model reads or writes is silently re-segmented before the model processes it: *loneliness* becomes `lone|li|ness`, and under current production vocabularies a line-final comma fuses with the newline into one token, so the line break, the one formal feature every poem has, stops being a boundary at all. We measure this intervention at scale: $\sim$3.27 million lines of public-domain verse, eight production tokenizers, four prosodic units (syllable, foot, rhyme, line), against a matched random-segmentation null model. Byte-BPE sits exactly at chance on syllables and feet; it protects rhyme spans through learned suffix morphology yet delivers rhyme as a shared token identity in under 1% of perfect rhyme pairs; and 72.6% of line endings are destroyed by a single dated regex clause from 2022. We trace that clause's provenance, report two corrective negative results (formal verse is *not* tokenized more harshly than free verse, and token-visible rhyme does *not* measurably improve rhyme completion in open models), then invert the analysis into practice: machine-verified verse forms defined over tokens, which fail to survive re-segmentation under any other vocabulary. The tokenizer, we argue, exercises prosodic agency that no poet delegated and no engineer intended. Code, pipeline, and poems: `https://github.com/AIscend-Research/tokenizer-prosody-creative`.

1 Introduction

A tokenizer is a segmentation decision made before any prompt is written: it fixes, once and in advance, which formal units can exist in a model's representation of a text. Verse makes this legible, because verse has explicit, externally defined units (syllable, foot, rhyme, line) measurable directly against token boundaries. This paper performs that measurement, locates its most consequential result in a single dated design decision, and takes the finding up constructively by composing verse in the tokenizer's own prosody.

The argument runs in six steps. (1) A tokenizer fixes which formal units are representable, and (2) verse makes this measurable. (3) Measured at scale, the units mostly do not align, and the line, the one unit every poem has, is not represented at all under current production OpenAI vocabularies. (4) The misalignment is a specific, dated decision, locatable in one regex clause, made for compression. (5) The tokenizer is therefore an *author*: it exercises prosodic agency no poet delegated and no engineer intended. (6) That authorship can be taken up rather than only suffered: we compose token-native forms whose failure to survive re-segmentation proves the vocabulary is constitutive of the form, not a container for it.

Contributions.

1. The first at-scale measurement of BPE segmentation against poetic units: 3.27M lines, 8 production tokenizers, syllables, feet, rhymes and lines, with a matched null model.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

2. The identification and provenance of a single regex clause removing lineation from $\sim$73% of English verse line endings, with an exact factorisation of the loss.
3. Two corrective negative results: formal verse is **not** tokenized more harshly than free verse (the apparent effect was one 14th-century poet), and token-visible rhyme does **not** measurably improve rhyme completion in open models.
4. A constructive inversion: token-native verse forms with a machine verifier, and the finding that these forms do not survive re-segmentation.
5. Released code and a fully reproducible pipeline (`https://github.com/AIscend-Research/tokenizer-prosody-creative`).

2 Related work

Tokenization as a design choice. Subword segmentation entered neural MT with BPE [1] and reached production models as byte-level BPE [2]. A growing literature documents its downstream costs in arithmetic [3], morphology [4, 5], and multilingual fairness [6, 7], and studies tokenizer construction itself [8]. That literature asks what tokenization costs *accuracy*; this paper asks what it costs *form*. **Computational poetics.** Automated scansion and syllabification have long histories [9, 10]; our human-unit layer builds on CMUdict [11] via `pronouncing` [12]. We measure alignment between prosodic and token boundaries rather than proposing a better scanner; prior work on LLM poetry has noted rhyming difficulty and its subword connection [13, 14]. **Constrained poetry.** The token-native forms of §6 belong to the Oulipo lineage [15], but are defined *over the representation itself*, unlike neural generation under human forms [13]. **Infrastructural authorship.** The claim that authorship can be embedded in infrastructure follows critical studies of classification and preprocessing as carriers of agency [16, 17].

3 Method

Corpora. Two public-domain sources, used as found; nothing is synthesised. **PoetryDB** [18]: 2,526 poems, 127 poets, 180,821 lines with structure intact; carries all form-level analysis. **Gutenberg Poetry Corpus v001** [19]: 3,085,117 lines for corpus-scale statistics; poem boundaries do not survive its extraction, so it carries no form-level claims. PoetryDB is a skewed sample (its top ten poets supply 54% of its lines and Chaucer alone 14%), which §4 shows matters; every per-poet or per-form figure therefore carries its support.

Tokenizers. Eight production tokenizers behind one interface returning tokens as *character spans* into the source text: a syllable, a foot and a rhyme are all character spans, so a token must be one too. Byte-BPE: `cl100k_base`, `o200k_base`, `p50k_base`, `r50k_base`, GPT-2, GPT-NeoX-20B. BERT WordPiece and flan-T5 SentencePiece normalise whitespace away before segmenting, placing boundaries at word edges by construction; they are kept as marked contrast rows, never pooled. Offset-reporting differences among HuggingFace byte-BPE models are normalised in the wrapper.

Human units. **Syllable counts** come from CMUdict (95.9% of word instances; out-of-dictionary words fall back to a flagged heuristic). OOV words are dominated by archaic elisions (*o'er*, *seem'd*), the same words BPE fragments hardest, so the dictionary gap and the fragmentation signal are correlated. **Syllable boundaries** are a convention English orthography does not mark; we implement two (Maximal Onset, and a hyphenation convention at 81.3% exact-match on a 155-word gold set) and run everything under both: placement-independent metrics move by exactly 0.000, and only syllable- and foot-boundary agreement is sensitive, its spread being its error bar. **Stress** treats monosyllables as promotable/demotable, as English metrics does. **Perfect rhyme** is an identical CMUdict rhyming part; refrains are excluded (identical words tokenize identically, which would flatter the tokenizer). A rhyme's span starts at the nucleus of the last stressed syllable: *night* contributes `ight`.

Metrics. Two complementary rates per unit: **integrity** (units lying wholly inside one token) and **boundary agreement** (prosodic boundaries that are also token boundaries), scored word-internally only, since whitespace is a hard BPE barrier every tokenizer gets nearly free. The strongest rhyme condition is `rhyme_shared_final_token`: both rhyming words end in the *same token id*, making the rhyme an identity in token space rather than a pattern to reconstruct from fragments. All

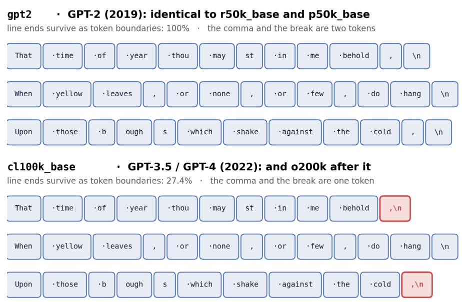

Figure 1: Three lines of Sonnet 73 tokenized whole under `gpt2` and `cl100k_base`, generated from the real tokenizers (· marks a space, `\n` a newline). Under `cl100k` a line-final mark absorbs the newline into one symbol (red): the break exists in the letters but not in the representation. Line 2 ends unpunctuated and survives under both: the loss is punctuation-conditional, not general.

aggregation is ratio-of-sums over stored numerators and denominators, never averaged per-poem rates. Poems are tokenized *whole*, newlines included, exactly as a model receives them; line-by-line tokenization would silently repair the very merges this project counts.

Controls. **Matched null model:** an agreement rate means nothing without a chance level, so the null segmenter is matched to the real tokenizer on pieces per word and on every boundary outside words, with only cut *positions inside words* randomised (1,200 poems $\times$ 3 seeds); a uniform-random-position baseline would be a straw man, shredding words BPE keeps whole. **Bootstrap intervals:** 2,000 resamples of *poems*, not lines, since lines within a poem share one poet's habits and one period's spelling. **Deconfounding** (§4) separates form from period; **decomposition** (§4) separates the poets' contribution to the line-break loss from the tokenizer's.

4 Results

The lineage: three generations of nothing, then a step. Running the OpenAI encoding sequence in release order turns the central finding into a dated event (Table 1, Figure 2). The first three encodings are identical to three decimals on every metric in this study; they share a pre-tokenizer regex. Then `cl100k` ships and line-end survival drops from 100% to 27.4% in a single release, and stays there: not a drift, a step function with a date on it. Meanwhile vocabulary quadrupled from 50k to 200k while polysyllable survival moved from 61.5% to 62.8%: buying four times the vocabulary bought poetry 1.3 percentage points. The compression objective and the prosodic objective are close to orthogonal, and only one was ever named.

The line break, decomposed. The 72.6% loss mixes two causes, how poets punctuate and what the tokenizer does about it, and they separate exactly:

$$P(\text{break destroyed}) \;=\; P(\text{line ends in punctuation}) \times P(\text{fused} \mid \text{punctuated}) \;=\; 73.9\% \times {\sim}99.7\%.$$

Unpunctuated line ends (26.1% of all) survive at **100% under every tokenizer**; punctuated line ends survive at 0.4% under `cl100k` and 100% under `gpt2`, every mark fusing alike. The poets supply the first factor; the tokenizer supplies the second as a near-certainty. One unintended form-level consequence: blank verse punctuates only 60.9% of its line ends and keeps the most lineation; quatrains punctuate 81.3% and keep the least. Which verse survives tokenization is decided by a punctuation habit interacting with a regex.

The rhyme asymmetry. Under `cl100k`, 96.7% of rhyme spans survive intact inside a single token and only 6.0% of perfect rhyme pairs are severed, but only **0.9%** of pairs end in the *same token id* (45,799 pairs of support). The vivid story, rhymes shredded by the tokenizer, is mostly false. What happens is quieter: *the model receives an intact rhyme carrying no symbolic mark that it is one* (Table 2).

encoding	shipped with	year	line ends	polysyll.
`gpt2`	GPT-2 (50,257)	2019	**100%**	61.5%
`r50k_base`	GPT-3 (50,257)	2020	**100%**	61.5%
`p50k_base`	Codex (50,281)	2021	**100%**	61.5%
`cl100k_base`	GPT-3.5/4 ($\sim$100k)	2022	**27.4%**	60.9%
`o200k_base`	GPT-4o ($\sim$200k)	2024	**27.2%**	62.8%

Table 1: The lineage: three generations identical to three decimals on every metric in the study, then one release removes lineation. Vocabulary size in parentheses. PoetryDB, 180,821 line ends.

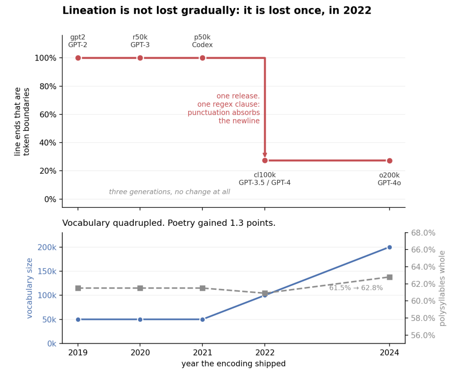

Figure 2: Top: line-end survival by release year, flat for three generations, then a single-release cliff to 27%. Bottom: vocabulary quadrupled; polysyllable survival moved 1.3 points. Dates are launch attributions.

Table 2: Headline rates with 2,000-resample bootstrap intervals (resampling poems, not lines) and support (`cl100k`, PoetryDB). Token-visible rhyme is under 1.1% at the top of its interval.

metric (`cl100k`)	rate	95% CI	support
words whole	0.8520	[0.8429, 0.8596]	1,354,483
polysyllables whole	0.6091	[0.5951, 0.6211]	308,380
syllable cuts matched	0.0941	[0.0909, 0.0975]	386,760
foot cuts matched	0.0376	[0.0362, 0.0391]	557,685
rhyme pairs severed	0.0604	[0.0564, 0.0644]	45,799
rhyme = shared final token	**0.0092**	**[0.0079, 0.0107]**	45,799
line ends that are token boundaries	0.2745	[0.2600, 0.2888]	180,821

Compared to what? The matched baseline. Figure 3 answers the question every agreement rate raises. On syllables and feet, BPE sits **exactly at chance**: every ratio between $1.00\times$ and $1.06\times$, with feet marginally *better* under random cutting ($0.94\times$). The tokenizer's within-word cuts carry essentially no information about syllabic structure: where it breaks a word is, as far as a poet's units are concerned, arbitrary. On rhyme, BPE beats chance substantially, severing rhyme spans half as often ($0.56\times$) and making rhymes token-visible $3.14\times$ more often, because it learns suffix morphology and rhyme lives on word endings. The tokenizer is *accidentally sympathetic* to rhyme, but $3\times$ almost nothing is still under 1%: 99.1% of perfect rhymes remain invisible as token identities.

Negative result I: form does not predict fragmentation. Raw, formal verse appeared to do slightly better on word integrity: formal$-$free $= +0.0329$ [$+0.0095$, $+0.0573$]. The effect does not survive a control: PoetryDB's formal poems are disproportionately *old*, archaic spelling is what BPE fragments, and Chaucer alone is 14% of the corpus. Removing that one poet collapses the difference to $+0.0008$ [-0.0066, $+0.0078$]; low-archaism poems (OOV $< 5\%$) give $+0.0012$; and `rhyme_shared_final_token` *changes sign* across treatments. The honest conclusion is a null: **formal and free verse are tokenized alike; vocabulary age, not form, predicts fragmentation.**

Negative result II: token-visible rhyme does not measurably help models rhyme. The representational finding predicts that models should complete token-visible rhymes better. We test this as a forced choice between the true rhyme word and a distractor matched on token count and frequency band, the invisible control group matched the same way; model and tokenizer are always paired (open weights, CPU, no API).

This is a null. The gap narrows as models improve ($+0.064 \to +0.035 \to -0.004$), driven entirely by better models improving on the *invisible* cases, consistent with capable models reconstructing rhyme from fragments; but three noisy points make that a hypothesis, not a finding. `cl100k`/`o200k` cannot be tested this way: no openly scoreable model uses them. **No causal claim from representa-**

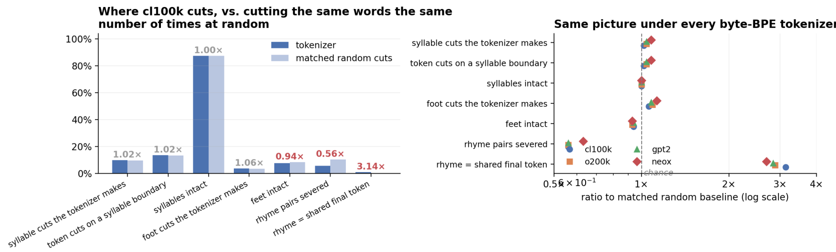

Figure 3: Left: `cl100k` vs. a null matched on pieces per word and every boundary outside words; only within-word cut positions are randomised (1,200 poems × 3 seeds). Right: the same ratios under every byte-BPE tokenizer. At chance on syllables and feet; 0.56×/3.14× on rhyme, both nearly zero in absolute terms.

Table 3: Rhyme-completion accuracy on token-visible vs. token-invisible perfect rhymes. All intervals span or sit on zero; the effect does not replicate.

model	tokenizer	visible	invisible	difference
GPT-2 (124M)	`gpt2`	0.930	0.866	$+0.064$ [$+0.000$, $+0.128$]
GPT-2-medium (355M)	`gpt2`	0.930	0.895	$+0.035$ [-0.023, $+0.093$]
Pythia (410M)	`neox`	0.929	0.933	-0.004 [-0.049, $+0.041$]

tion to model output is available; the tokenizer's authorship concerns which forms are representable, not model competence. Models rhyme well; they rhyme in a prosody nobody chose.

5 Provenance: naming the decision

The track's theme asks where creative agency begins and who is accountable when it is distributed; this project answers with a measurement and a named location. The mechanism of §4 is one clause in `cl100k`'s pre-tokenizer regex: the pattern grouping a run of punctuation with trailing whitespace-and-newlines, so that `,+\n` becomes one vocabulary item. Introduced with `cl100k_base` in 2022 and inherited by `o200k_base`, its rationale is compression: punctuation-plus-newline is frequent in web text, and merging it saves a token at almost every sentence end.

Four claims about agency follow, made explicitly. **Agency was exercised, not absent**: a prosodic choice was made (punctuation absorbs the newline after it) with a date, an artifact and a rationale; it was simply not made *as* a prosodic choice. **Agency was displaced, not destroyed**: the poet still chooses the words; the tokenizer chooses whether the line break exists; authorship over the poem's form is split between two parties, one of whom never intended to participate. **Accountability does not reduce to intent**: nobody decided poems should lose their lines, and the effect is real regardless: distributed agency with no decision-maker to point at, only a clause and a compression objective optimised for years while the prosodic objective was never named. **Agency can be reclaimed by constraint** (§6). We do not overclaim harm: this is not hostility to verse but blindness, plus one accidental sympathy.

6 The inversion: writing in the machine's prosody

If the tokenizer is a prosodist, these are its forms, each promoting a feature of BPE segmentation into a formal rule: the *isotokon* (every line exactly n tokens), the *tokenku* (5/7/5 in tokens), *token-trochaic* metre (every word exactly two tokens, alternating word-initial and word-continuing symbols STRONG–weak), *token-rhyme* (two lines end in the same token id), and the *clean break* (every line break a token boundary, not automatic under `cl100k`). All poems are machine-verified against the text tokenized whole, as a model receives it.

The vocabulary imposes a central asymmetry before any poem is written: a word that is a single token can only token-rhyme with itself, so every non-trivial token-rhyme class consists entirely of words the tokenizer has already broken: *to rhyme in the machine's prosody you must use words it cannot keep whole.* Under `cl100k`, 33.0% of word *types* are a single token but 86.6% of word *instances* are; that leaves 1,379 non-trivial token-rhyme classes over 11,921 words, the largest being `-'d` (101 words: *call'd*, *seem'd*), `-'s`, `-ly`, `-ing`. Token-rhyme is richest where BPE fragments hardest: archaic spelling and elision. Three of the composed poems follow, in full; each is a poem *for* `cl100k`.

The Invisible Prosodist

Some engineer I never met had decided
where my breath would fall and stop
He fixed the cuts before I wrote
The rules were set before the poem
The line break is not a boundary
A comma and a newline are one
I write inside a room he built
Nothing I do can move that seam

Eight lines of exactly seven tokens; in syllables they run 9/7/8/8/8/8/8/8, a strict form only one reader can perceive. Line 6 states a measured fact.

Rhymes You Cannot Hear

Time goes over me quick and winged `[ed]`
something in the dark is watching sly `[ly]`
All that I gathered in was scorned `[ed]`
and all I built came down lowly `[ly]`

Four lines of exactly eight tokens, rhymed *abab* on final token ids, printed beside each line. To the model these are exact rhymes, the same final symbol; to a reader they do not rhyme at all. The converse holds far more often: under 1% of PoetryDB's perfect rhymes are visible to `cl100k` this way.

Token-Trochaic Quatrain

sleepless darkening stillness
quietness deepening voiceless
unspoken quickening softness
unbroken shapeless measureless

Every word costs exactly two tokens, so word-initial and word-continuing symbols alternate STRONG–weak throughout. The form imposed a consequence the author did not choose: *the*, *of* and *and* each cost one token, so a token-trochaic poem cannot contain grammar; it is necessarily a list. That unchosen consequence is the paper's clearest small demonstration of delegated agency. We also assemble *found* token-verse: published lines already satisfying the forms, selected from a 410,275-line pool by nothing but the tokenizer: poems with two authors, one of whom never intended to write.

poem	`cl100k`	`o200k`	`gpt2`	`neox`
Tokenku	✓	✓	×	×
The Invisible Prosodist	✓	✓	×	×
Token-Trochaic Quatrain	✓	×	×	×
Rhymes You Cannot Hear	✓	×	×	×
Found Token-Sonnet	✓	×	×	×
Found Isotokon	✓	✓	✓	×

Table 4: Re-segmentation survival of the composed forms. Same words, no longer the same form.

Table 4 is the constructive half's evidence: the vocabulary is not a neutral container for the poem but a condition of its existence, which is what makes segmentation an authorial act, and agency asserted by writing *inside* it.

7 Limitations, ethics, and conclusion

(1) PoetryDB is skewed (top ten poets = 54% of lines, Chaucer 14%), so corpus-wide figures over it are not claims about English verse in general, though the 3.02M-line Gutenberg corpus agrees within a point on every corpus-scale metric. (2) CMUdict is modern American English applied to largely pre-20th-century verse: historical rhymes are missed and elided forms are out-of-dictionary, making rhyme counts conservative. (3) No causal claim from representation to model output is supported: the behavioural test is a null covering only `gpt2`/`neox`. (4) The form classifier is unvalidated (unlike the syllabifier), and foot-level figures rest on automated scansion, the softest in the set. Both corpora are public domain and used as found; the analysis names a documented artifact and its stated compression rationale, not an intent.

Reproducibility. All tokenizers run locally, the behavioural test uses open weights on CPU, and there are no API calls. `./run_all.sh` reproduces every number from a clean checkout; 57 tests cover the measurement layer; the producing commit is pinned at `https://github.com/AIscend-Research/tokenizer-prosody-creative`.

Conclusion. The tokenizer is an invisible prosodist: an author embedded in infrastructure whose segmentation decisions predate any prompt. It is blind to syllable and foot, accidentally kind to rhyme, and, since one regex clause in 2022, it dissolves the line break for three-quarters of English verse. Nobody chose this as prosody, which is why it matters for creative agency: the choice was made anyway. The composed poems show the other direction is open; their failure to survive re-segmentation proves it: the vocabulary does not carry the poem; it co-writes it.

References

[1] Sennrich, R., Haddow, B. & Birch, A. (2016) Neural machine translation of rare words with subword units. In *Proc. ACL 2016*, pp. 1715–1725.

[2] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. & Sutskever, I. (2019) Language models are unsupervised multitask learners. OpenAI technical report.

[3] Nogueira, R., Jiang, Z. & Lin, J. (2021) Investigating the limitations of transformers with simple arithmetic tasks. *arXiv:2102.13019*.

[4] Bostrom, K. & Durrett, G. (2020) Byte pair encoding is suboptimal for language model pretraining. In *Findings of EMNLP 2020*, pp. 4617–4624.

[5] Hofmann, V., Pierrehumbert, J. & Schütze, H. (2021) Superbizarre is not superb: Derivational morphology improves BERT's interpretation of complex words. In *Proc. ACL 2021*, pp. 3594–3608.

[6] Petrov, A., La Malfa, E., Torr, P. & Bibi, A. (2023) Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems 36*.

[7] Rust, P., Pfeiffer, J., Vulić, I., Ruder, S. & Gurevych, I. (2021) How good is your tokenizer? On the monolingual performance of multilingual language models. In *Proc. ACL 2021*, pp. 3118–3135.

[8] Mielke, S. J., Alyafeai, Z., Salesky, E., Raffel, C., Dey, M., Gallé, M., Raja, A., Si, C., Lee, W. Y., Sagot, B. & Tan, S. (2021) Between words and characters: A brief history of open-vocabulary modeling and tokenization in NLP. *arXiv:2112.10508*.

[9] Agirrezabal, M., Alegria, I. & Hulden, M. (2016) Machine learning for metrical analysis of English poetry. In *Proc. COLING 2016*, pp. 772–781.

[10] Anttila, A. & Heuser, R. (2016) Phonological and metrical variation across genres. In *Proc. Annual Meetings on Phonology*.

[11] Weide, R. (1998) *The CMU Pronouncing Dictionary*. Carnegie Mellon University. `http://www.speech.cs.cmu.edu/cgi-bin/cmudict`.

[12] Parrish, A. (2015) *pronouncing*: A simple interface for the CMU Pronouncing Dictionary. `https://pypi.org/project/pronouncing/`.

[13] Lau, J. H., Cohn, T., Baldwin, T., Brooke, J. & Hammond, A. (2018) Deep-speare: A joint neural model of poetic language, meter and rhyme. In *Proc. ACL 2018*, pp. 1948–1958.

[14] Ormazabal, A., Artetxe, M., Agirrezabal, M., Soroa, A. & Agirre, E. (2022) PoeLM: A meter- and rhyme-controllable language model for unsupervised poetry generation. In *Findings of EMNLP 2022*, pp. 3655–3670.

[15] Motte, W. F. (ed.) (1986) *Oulipo: A Primer of Potential Literature*. Lincoln: University of Nebraska Press.

[16] Bowker, G. C. & Star, S. L. (1999) *Sorting Things Out: Classification and Its Consequences*. Cambridge, MA: MIT Press.

[17] Dourish, P. (2016) Algorithms and their others: Algorithmic culture in context. *Big Data & Society* **3**(2).

[18] PoetryDB (2024) An API for internet poets. `https://poetrydb.org`.

[19] Parrish, A. (2018) *A Gutenberg Poetry Corpus*, v001. `https://github.com/aparrish/gutenberg-poetry-corpus`.

[20] OpenAI (2022) *tiktoken*: A fast BPE tokeniser for use with OpenAI's models. `https://github.com/openai/tiktoken`.